Web Dashboard

03

Transforming raw data into actionable insights through real-time visualisation assists in monitoring performance, identifying and responding to changes, and simplifying complex data comprtehensible at a glance.

A dashboard is a visual tool that displays important information in a clear and organized way. It functions like a control panel, allowing users to quickly view and analyse key data.

For instance, a teacher managing a class needs to keep track of student information such as Student Grades, Attendance Records, Upcoming Assignments etc.

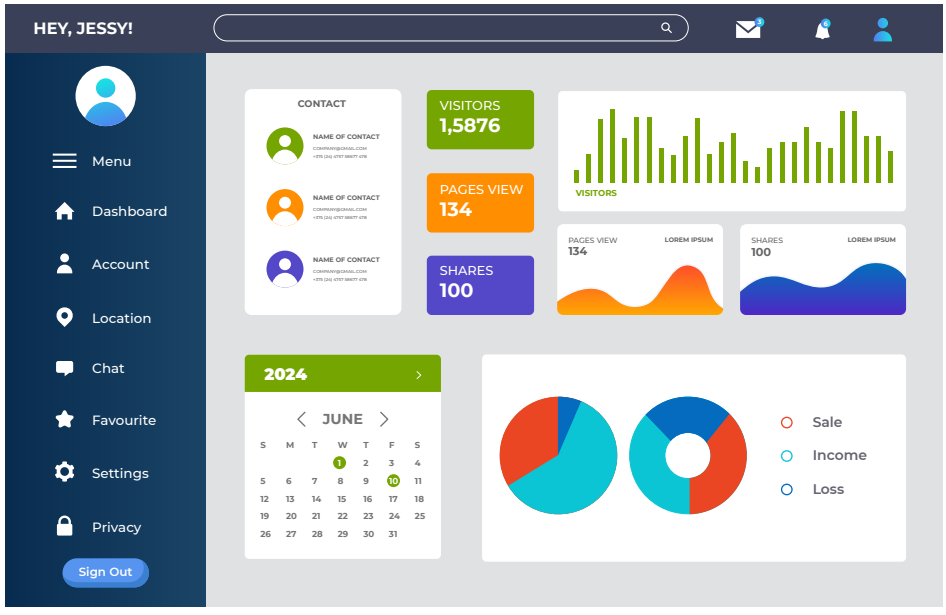
If a dashboard shows all of the above information, a teacher can easily view students' performance, track attendance, and keep an eye on upcoming tasks. It helps in managing student information efficiently and making informed decisions.

03. Web Dashboard

Every web application requires an Admin Dashboard for analysing and maintaining the internal details of an organisation. The admin panel supports user-related functions, such as providing insight into user behaviour, handling profiles that violate the site's terms and conditions, tracking transactions, and more.

Such dashboards are used across various fields like finance, engineering, science, and commerce, where data visualisation is crucial. IoT is another recent technology where dashboards are extremely popular.

Below are sample dashboards commonly seen online.



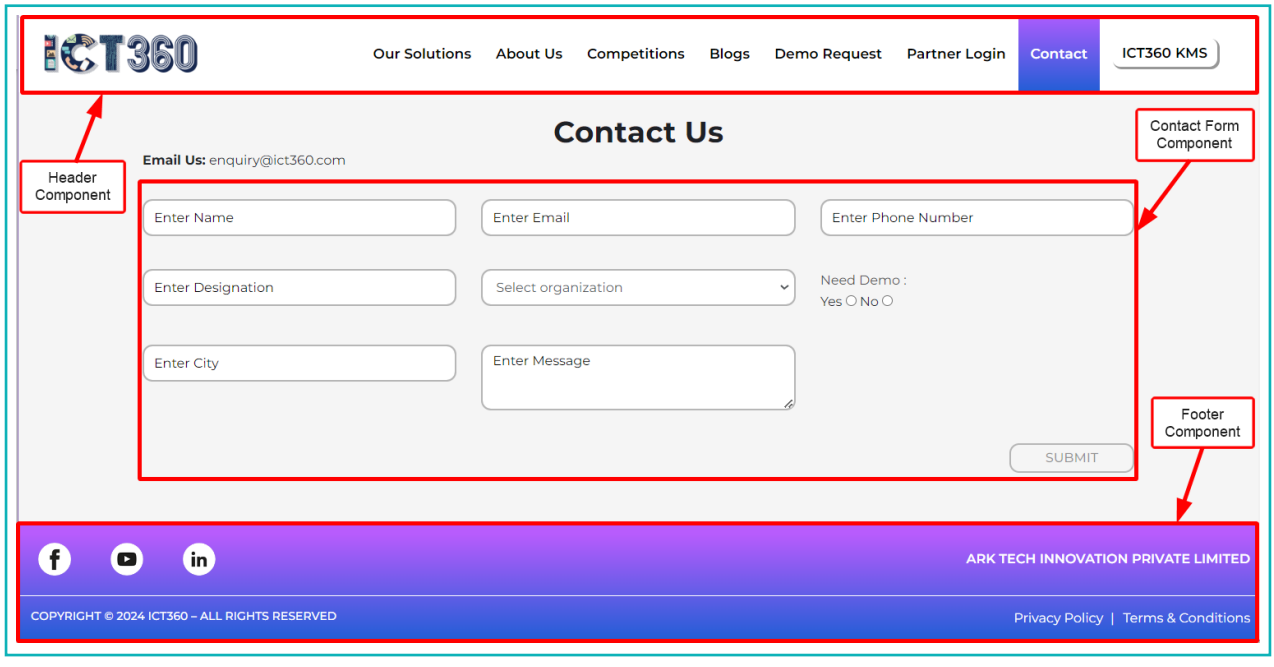
Such dashboards can be created with the help of React.js.

What is React?

React is a widely popular JavaScript library developed by Facebook, primarily used for building user interfaces. As an open-source, component-based library, React focusses solely on the view layer of applications, whether mobile or web.

React Components

As mentioned earlier, React is a component-based library, but what exactly are **Components**? Consider the following website, where we can identify parts of the page as Components.



- **Component-based design** refers to a design format in which the user interface is divided into several components, each having its own design and functionality.
- **Components** are the core building blocks of React applications and facilitate the development of user interfaces.
- It is also reusable. For example, a 'Navigation Bar' component can be created and used across multiple pages on a website or web application.
- Component-based design is effective for error detection because it allows debugging of only the specific component with an error, rather than troubleshooting the entire web application.

Getting Started

Before building a React App, Node.js needs to be installed on the system.

Node.js (or simply Node) is not a programming language. It is an open-source, cross-platform runtime environment that enables JavaScript code to be executed on the server. It also includes a package manager called **npm (node package manager)**, which is necessary for downloading the React library and creating a React project.

To download Node, visit https://nodejs.org/en/ and click the 'Download' button. Once downloaded, install it by following the simple on-screen instructions.

After installing Node, proceed with creating the React application.

Bytes

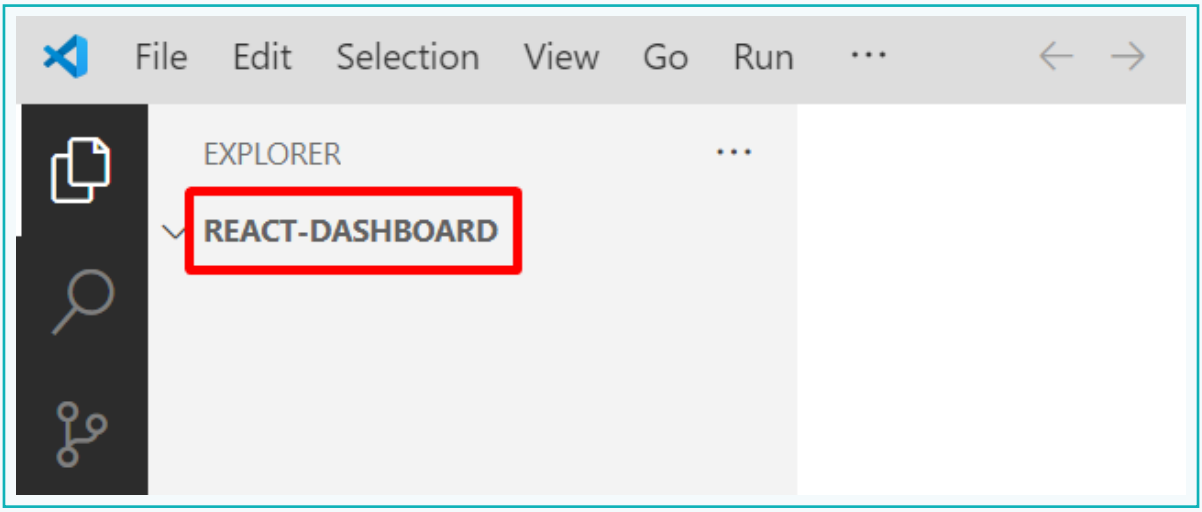
One of the most common uses of React is creating **Single Page Applications** or SPAs. An **SPA** is an application design where the entire web application is rendered on a single page, with new content appearing as the user scrolls down.

Activity

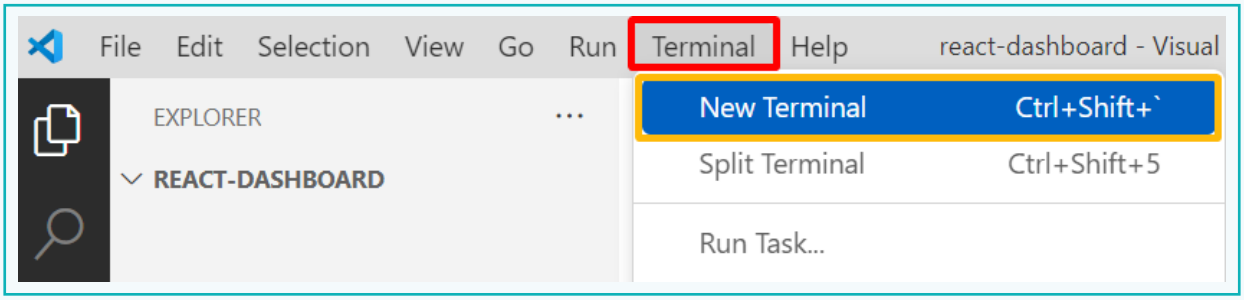
Create a 'Web Dashboard' using tools and technologies such as the REACT library. The purpose of this dashboard is to display and keep track of the sales and users associated with a particular organization.

Part 1 - Create a React Application

1. Create a folder named "react-dashboard". Open VS Code from the 'Start' menu. Then, go to the 'File' menu', select 'Open Folder', and choose the 'REACT-DASHBOARD' folder.



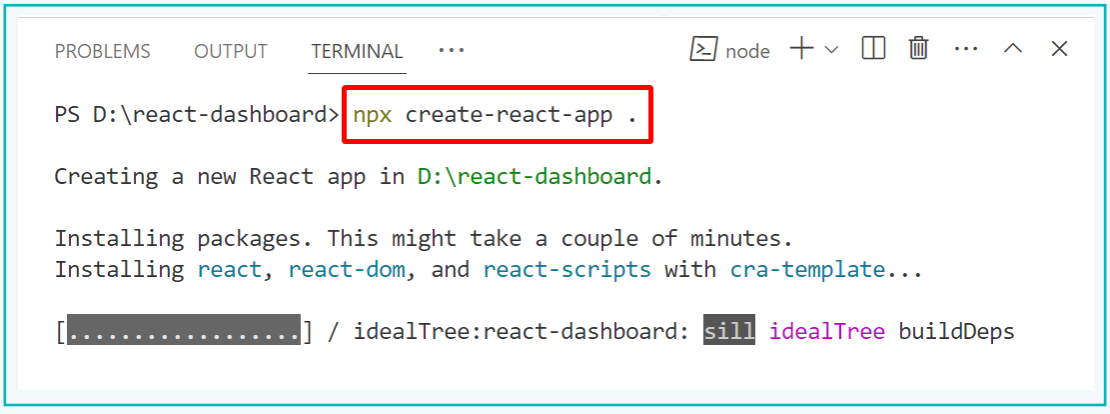
2. Navigate to the 'Terminal' tab on the menu bar and select 'New Terminal' to open the 'Terminal' in VS Code. The Terminal will appear at the bottom of the VS Code window.



Activity

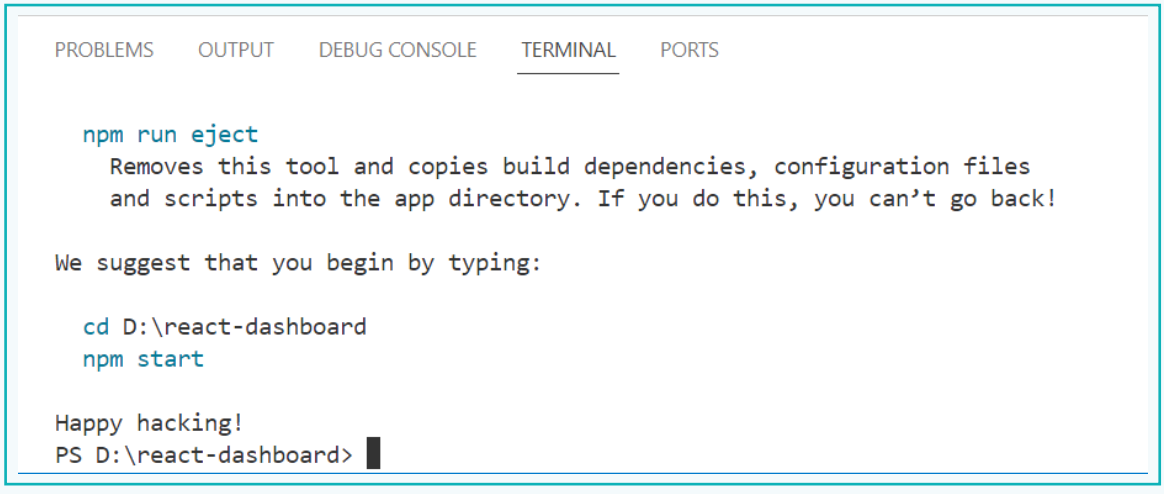
To create a React application, the **npx create-react-app** command needs to be executed. This command downloads all the necessary files and sets up the React project.

3 In the Terminal, type the following command, followed by a **.(dot)**– *npx create-react-app.*



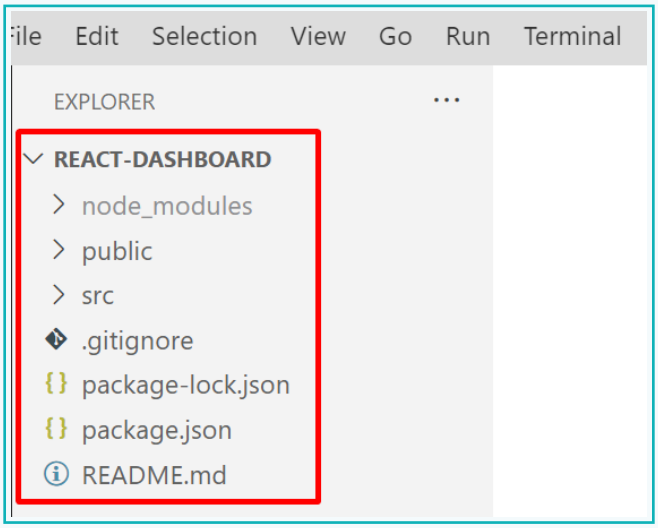
Note:
The **.(dot)** at the end of the command indicates that the React App should be created within the same folder. To create the app in a different folder, specify the desired folder name instead of the dot.

The installation process may take a few minutes, depending on the internet speed. Once it is finished, a message will appear in the terminal indicating that the React application has been successfully installed.



Activity

Observe that several files and folders have been created inside the 'react-dashboard' folder. These were created by the **npx create-react-app** command during the project setup.



Brief Description of Files and Folders

- **node_modules:** This folder contains all the project dependencies, including the react library and any other required libraries.
- **public:** This is a special directory that includes assets such as images, HTML files, icons, etc. It also contains the 'índex.html' file, which acts as the entry point for rendering the React application in the browser.
- **src:** This is where all the components of the application are located. Most of the work will be done here.

The key files in this folder include: –

a. **index.js** – This is the primary JavaScript file responsible for kick-starting the application, loading components, and injecting them into the HTML file 'index.html'.

b. **App.js** – This is the primary or root component file. All other components created must be included in this file.

c. This folder also contains CSS file (App.css) for design properties and test files used for testing.

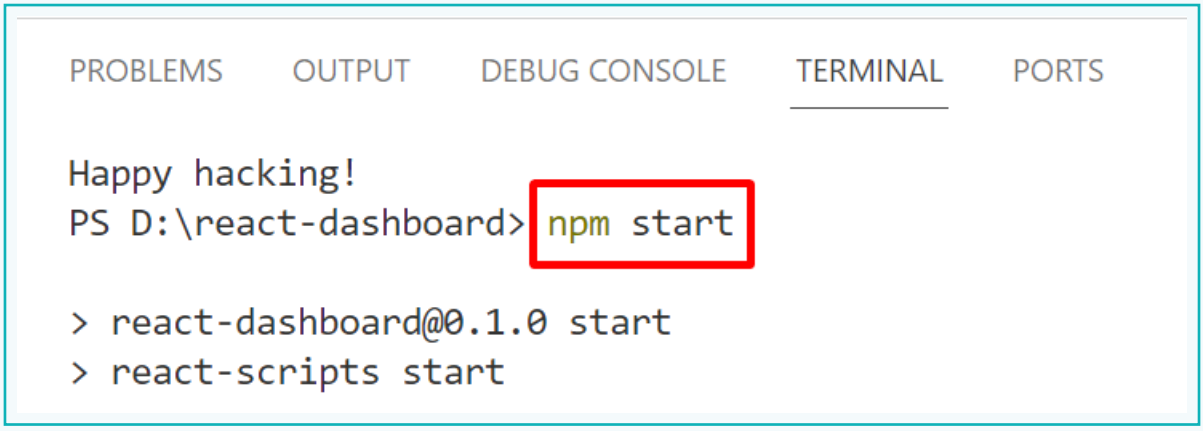
- **package.json:** This is a JSON file that contains the names of all the project dependencies, as well as other script files and commands.

The remaining files are application-related and are not significant at the moment.

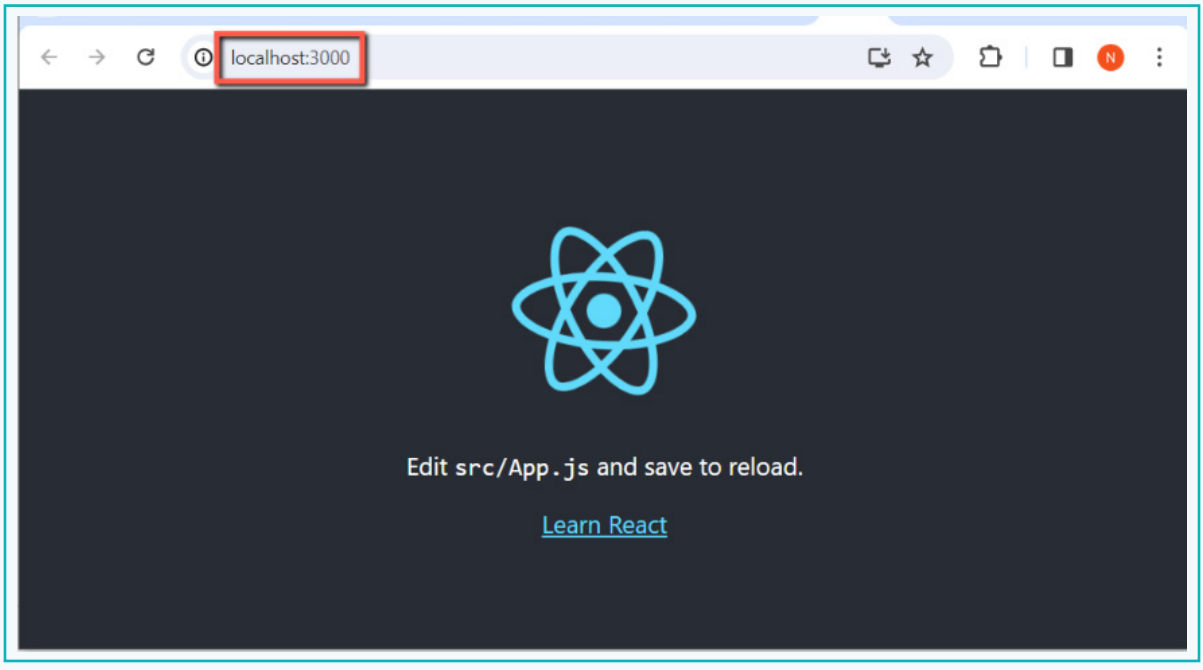
Activity

Part 2 – Execute the Application

1. Open the terminal and enter the following command – *npm start*. This command will launch a local server and execute the React application on *port 3000*.



2. The output will be displayed in the browser at http://localhost:3000. This is the default address where the React application runs when started with the *npm start* command.



Activity

Part 3 - Remove Unwanted Contents/Files from the Application

Every React web app contains some demo content that may not be required. To keep the web app clean and organised, follow below steps -

1. Open the 'index.html' file located inside the 'public' directory. Remove the unwanted *<link>* tags and comments within the *<head>* tag.
   The content to be removed is highlighted within the red border.

```
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
      Notice the use of %PUBLIC_URL% in the tags above.
      It will be replaced with the URL of the `public` folder during the build.
      Only files inside the `public` folder can be referenced from the HTML.

      Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
      work correctly both with client-side routing and a non-root public URL.
      Learn how to configure a non-root public URL by running `npm run build`.
    -->
    <title>React App</title>
  </head>
```

2. Similarly, remove the contents of the *<body>* tag, as indicated by the red border.

```
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
    <!--
      This HTML file is a template.
      If you open it directly in the browser, you will see an empty page.

      You can add webfonts, meta tags, or analytics to this file.
      The build step will place the bundled scripts into the <body> tag.

      To begin the development, run `npm start` or `yarn start`.
      To create a production bundle, use `npm run build` or `yarn build`.
    -->
  </body>
</html>
```

Activity

3 The ‘index.html’ file should look like this now. Notice how the page appears clean and organised.

index.html M

public > index.html > ...

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta name="description" content="Web site created using create-react-app" />
    <title>React App</title>
  </head>

  <body>
    <div id="root">

    </div>
  </body>
</html>
```

4 Next, open the ‘App.js’ file located in the ‘src’ directory and remove the contents highlighted by the red border. This content is responsible for displaying the default output. Since new content will be created, the default content should be removed.

App.js M

src > App.js >

```
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">

      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a className="App-link" href="https://reactjs.org" target="_blank"
          rel="noopener noreferrer" >
          Learn React
        </a>
      </header>

    </div>
```

Activity

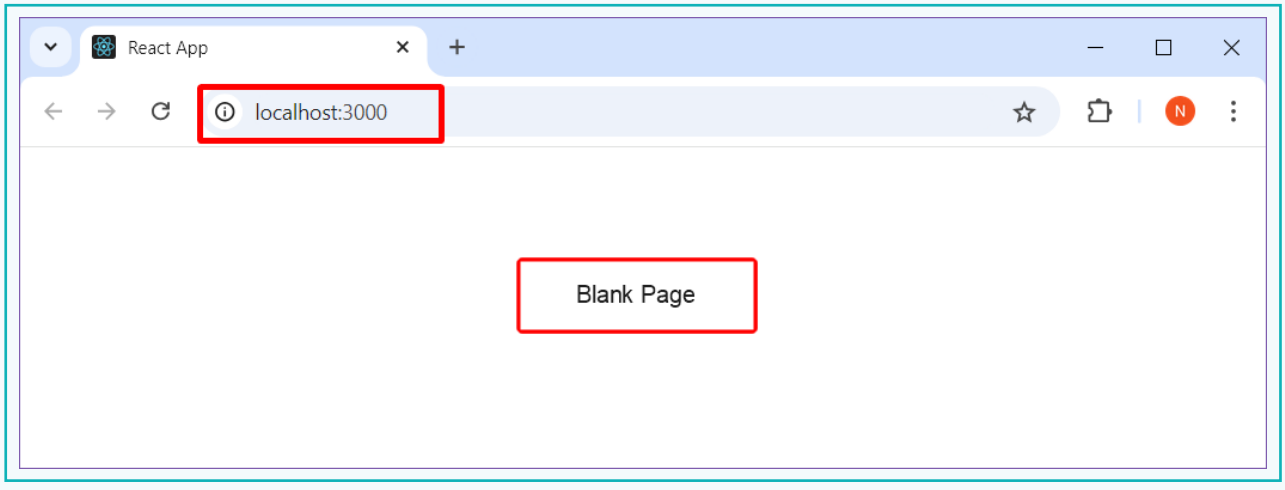
5 The figure below shows the 'App.js' file after deleting the header contents.

App.js M

src > App.js > ...

```
1   import './App.css';
2
3   function App() {
4     return (
5       <div className="App">
6
7       </div>
8     );
9   }
10
11  export default App;
```

6 Run the application. A blank page will appear in the browser. This occurs because the default content was removed from the 'App.js' file.

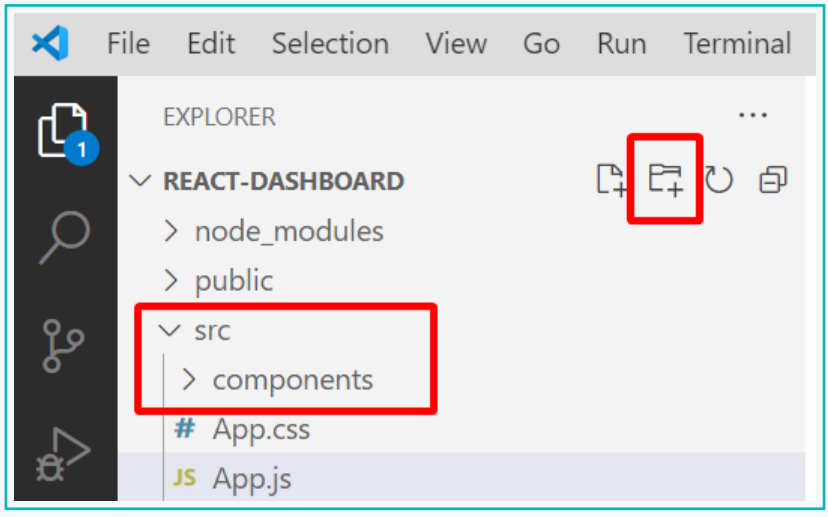


Activity

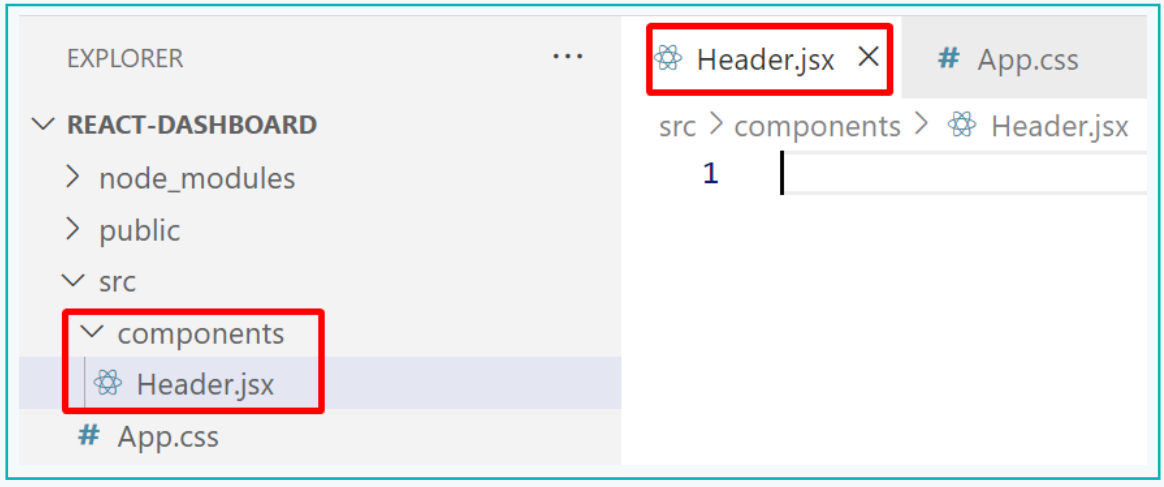
Part 4 – Create a Header Component

Depending on the structure of a website, it may contain several components. To store all the component files, create a folder called “components”. This also helps in keeping the components organised.

1. Create a folder called ‘components’ within the ‘src’ folder by selecting ‘src’ folder and clicking on the ‘New Folder’ icon present in the sidebar of VS Code.



2. Create a file “Header.jsx” within the ‘components’ folder. This file can also be considered as the *Header* component.



About JSX Files

Notice that the React application contains only a single HTML file, which is 'index.html'. All other files are primarily *JavaScript (.js)* files. So, when adding content to a component using HTML, it must be written inside a JavaScript file.

However, HTML code cannot be directly written within a JavaScript file since they both are different coding languages.

To address this, 'JSX' is required. **JSX** stands for **JavaScript XML,** and it allows writing HTML within JavaScript easily and conveniently.

Activity

3 Open the 'Header.jsx' file and add the React component boilerplate code. Every React component file has a fixed format consisting of:

- An *import* statement to include the React library.
- A *function* and a *return* statement, where the component is defined.
- An *export* statement to make the component available for use by other components within the application.

Header.jsx

src > components > Header.jsx > ...

```
1  import React from 'react';
2
3  function Header() {
4    return (
5      <div>
6
7      </div>
8    );
9  }
10
11 export default Header;
12
```

Activity

4 Add a *<h1>* heading with the title 'REACT DASHBOARD'. A different heading can be used if preferred.

```
3   function Header() {
4     return (
5       <div>
6
7         <h1> REACT DASHBOARD </h1>
8
9       </div>
10    );
11  }
```

5 Check the output on the browser.

The heading *React Dashboard* does not appear in the browser because a new component must be included in the 'App.js' file. The **App.js** file serves as the main parent component, with all other components acting as its child components.

Activity

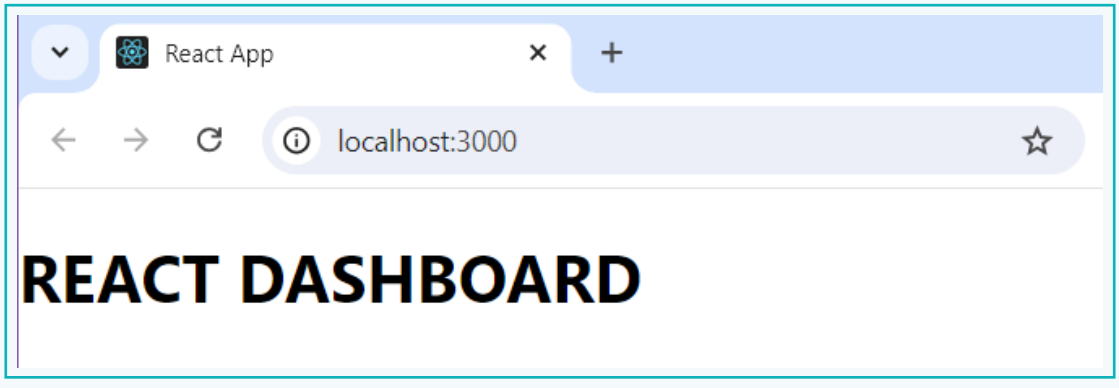
6. Open the 'App.js' file and import the *Header* component using the *import* statement.

```
App.js
src > App.js > ...
1   import React from 'react';
2   import './App.css';
3   import Header from './components/Header';
4
5   function App() {
6     return (
7       <div className="app">
8
9       </div>
10    );
11  }
12  export default App;
```

7. Include the *Header* component within the *App* function of the 'App.js' file, by writing its name like an HTML tag. This process is known as **Rendering a Component.**

```
App.js
src > App.js > ...
1   import React from 'react';
2   import './App.css';
3   import Header from './components/Header';
4
5   function App() {
6     return (
7       <div className="app">
8
9         <Header />
10
11      </div>
12    );
13  }
14  export default App;
```

8. Now, the output will be visible on the browser.



Activity

To summarise, after you create a component, you need to –

1. Import the component into the 'App.js' file.
2. Write the component name in an HTML tag format wherever it should appear on the webpage.

Part 5 – Design the Header Component with CSS

1 Before designing with CSS, assign a class to the * * tag in the 'Header.jsx' file.

Note:
Since *class* is a reserved keyword in JavaScript, the *className* attribute is used instead of the *class* attribute to apply CSS properties.

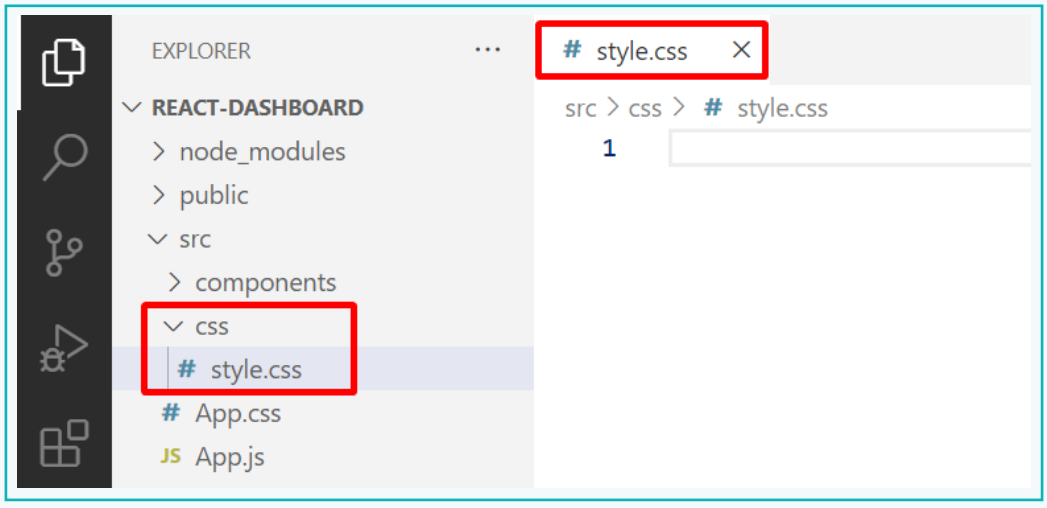
Header.jsx

src > components > Header.jsx > ...

```jsx
import React from 'react';

function Header() {
  return (
    <div className='header'>

      <h1> REACT DASHBOARD </h1>

    </div>
  );
```

Activity

2 Create a 'css' folder inside the 'src' folder and add a file named 'style.css'. This file will contain the CSS properties.



3 Define some basic CSS properties using the *wildcard (*)* operator and the *.header* class selector.

Activity

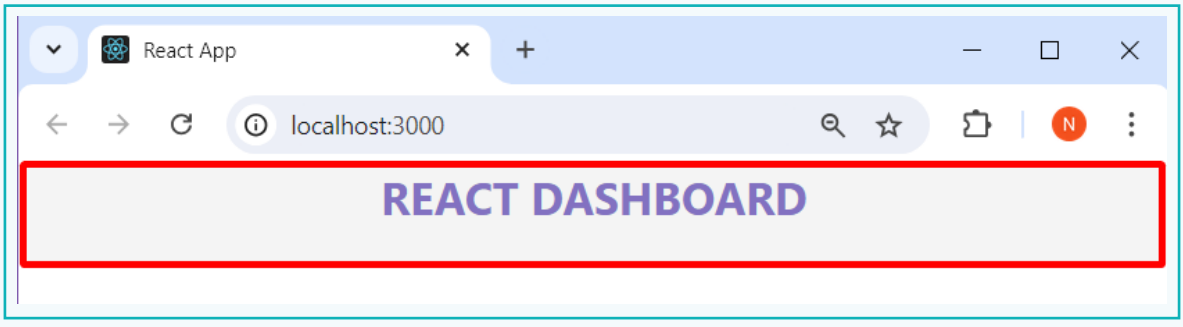
4 To apply the specified CSS properties to the 'Header.jsx' file, import the 'style.css' file into the 'Header.jsx' file.

style.css | Header.jsx

c > components > Header.jsx > ...

```
1   import React from 'react';
2
3   import '../css/style.css'
4
5   function Header() {
6     return (
7       <div className='header'>
8
9         <h1> REACT DASHBOARD </h1>
10
11      </div>
12    );
```

- The *Header* component will appear as shown below.

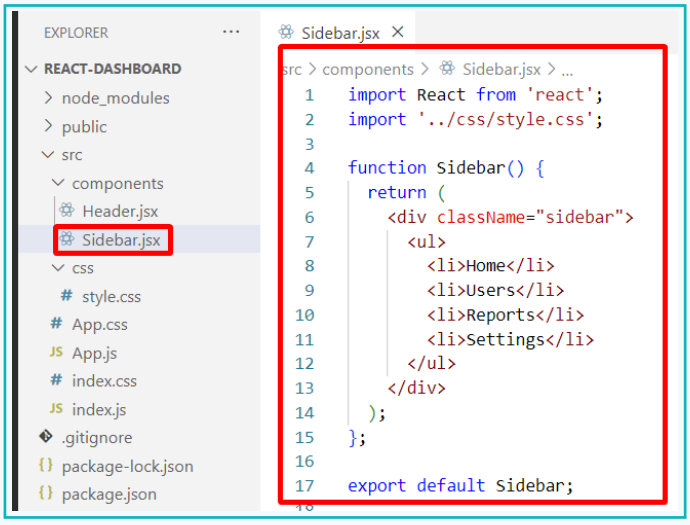


Activity

Part 6 – Create the Sidebar and Dashboard Components

Let us create another component named *Sidebar* and align it to the left of the webpage. The purpose of the sidebar is to facilitate navigation and provide users with quick access to various dashboard sections or pages.

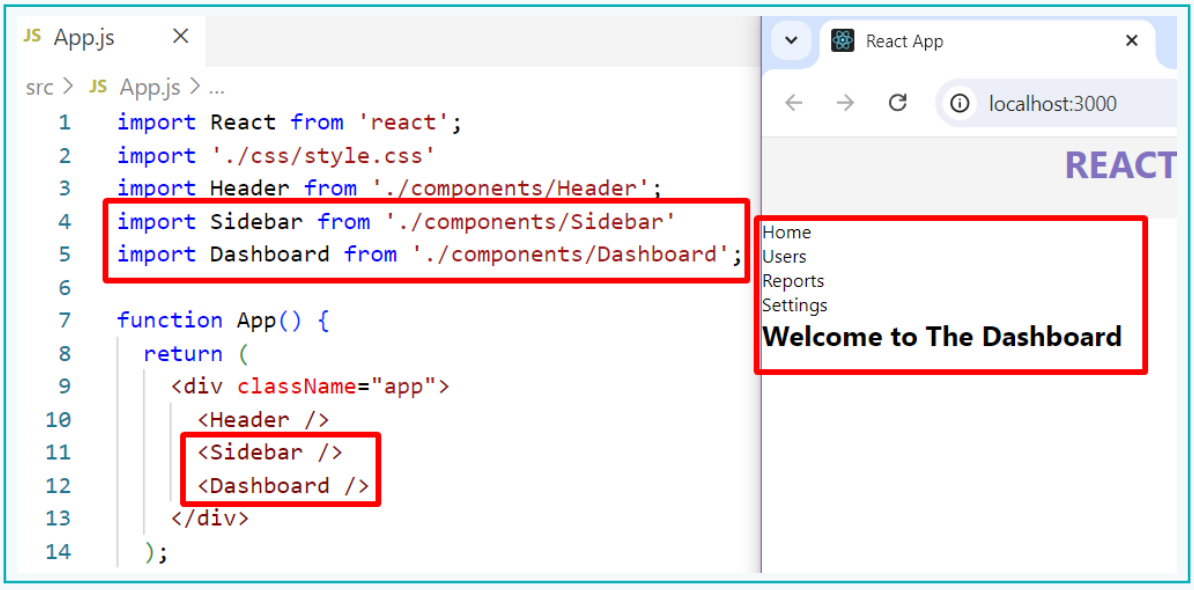
1. Create a 'Sidebar.jsx' file inside the 'components' folder. Add some navigation items in the form of an 'Unordered List' *(<ul>)*. Assign a class 'sidebar' to the * * to style this element with CSS.



2. Create another file 'Dashboard.jsx' inside the 'components' folder. This component will represent the main content area of the dashboard.

Activity

3 Import the *Sidebar* and *Dashboard* components in the 'App.js' file and use them, just like the *Header* component.



Note:
There is no need to import the 'App.css' file because we are using the CSS properties defined in the 'style.css' file.

Part 7 - Component Layout using CSS Flexbox

The *Sidebar* and *Dashboard* components appear vertically, one above another. To design the layout so that they are aligned horizontally, side by side, the simplest approach is to use **CSS Flexbox.**

Flexbox is a layout model in CSS that allows the flexible arrangement of items within a container, enabling both vertical and horizontal alignment.

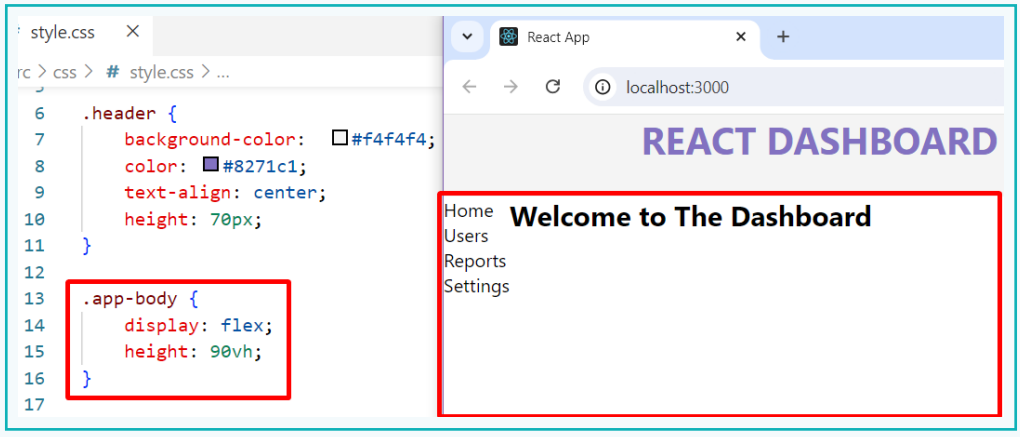
To start using Flexbox, first define a *Flexbox* container by setting the CSS *display* property of an HTML element to *flex*.

Activity

1. In the 'App.js' file, wrap the *Sidebar* and the *Dashboard* components in a * * container, which will be converted into a Flexbox. Also, assign a class *app-body* to it.

```
7   function App() {
8     return (
9       <div className="app">
10        <Header />
11        <div className='app-body'>
12          <Sidebar />
13          <Dashboard />
14        </div>
15      </div>
16    );
17  }
18  export default App;
```

2. Open the 'style.css' file and set the *display* property to *flex* for the *.app-body* selector. This will align the two components horizontally.
   - Additionally, set the *height* property to *90vh* so that it occupies 90% of the height of the browser.



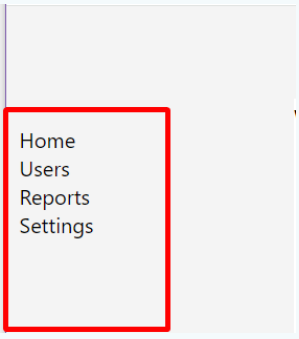
Activity

Part 8 – Design the Sidebar and Dashboard Components

Apply some basic design to the *Sidebar* and *Dashboard* components.

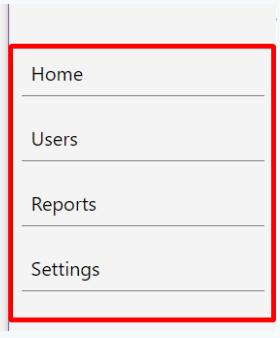
1. Using CSS, set a background colour. Add some spacing using the *padding* property and assign a fixed width to it using the *width* property.

```
.app-body {
    display: flex;
    height: 90vh;
}

.sidebar {
    background-color: #f4f4f4;
    padding: 20px 10px;
    width: 200px;
}
```



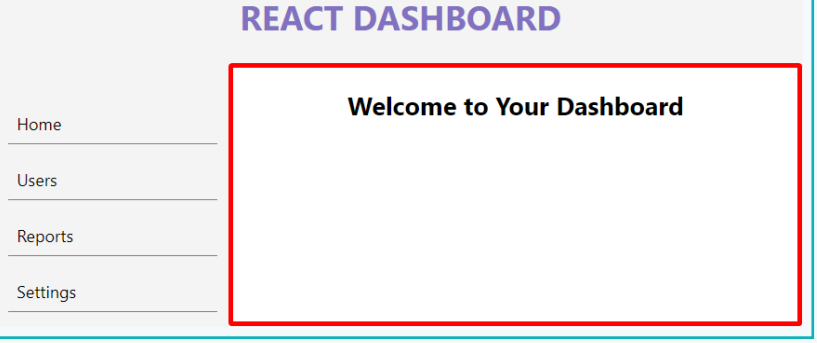
2. For the list items, create some spacing using the *margin* and *padding* properties. Add a border below the list items using the *border* property.

```
.sidebar {
    background-color: #f4f4f4;
    padding: 20px 10px;
    width: 200px;
}

.sidebar ul li {
    margin: 15px 0;
    padding: 8px;
    border-bottom: 1px solid grey;
}
```



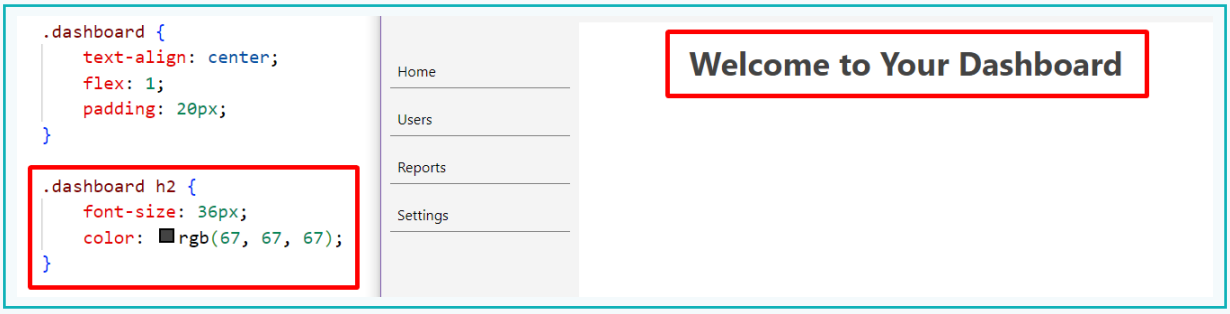
3. The 'Dashboard' component includes a single *<h2>* tag within the * * container. Apply *20px* padding to the container and align the content at the centre. Make this component occupy the entire space using the *flex: 1* CSS property.

```
.sidebar ul li {
    margin: 15px 0;
    padding: 8px;
    border-bottom: 1px solid
}

.dashboard {
    text-align: center;
    flex: 1;
    padding: 20px;
}
```

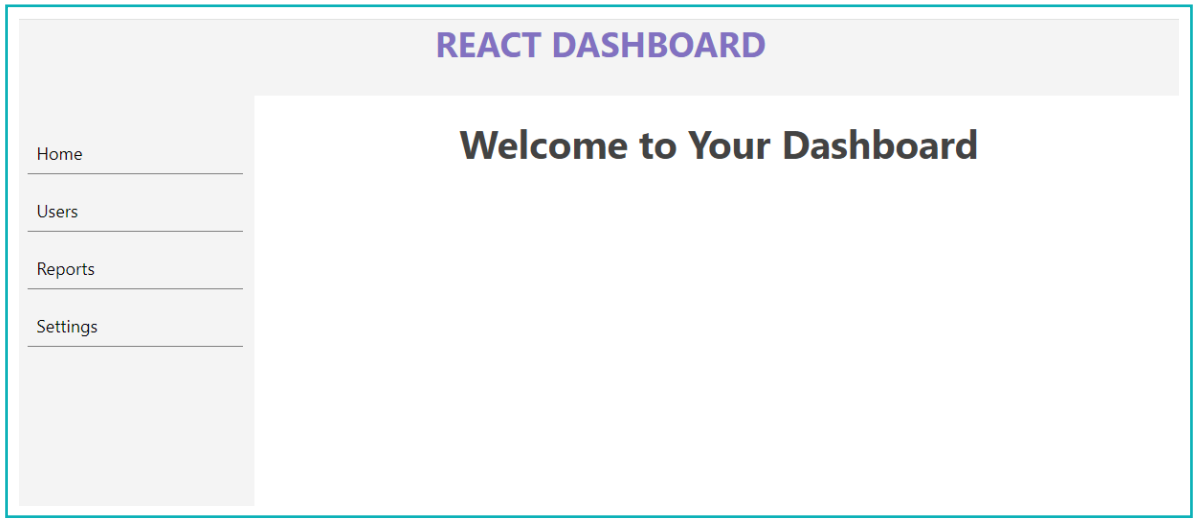


Activity

4 Increase the font size and change the font colour for the *<h2>* heading.



Output -



In this lesson, we accomplished building the basic dashboard components. In the next lesson, we will enhance it by adding visualizations to make it more interactive and visually appealing.

To summarize, we discovered that web dashboards are essential tools used across various fields, offering key insights and management capabilities. We understood that the React library is commonly used to build such dashboards due to its component-based architecture. Additionally, we created a dashboard using React.

Exercise

Fill in the Blanks.

1. The _____ file is referred to as the root component in a React application.
2. The index.html file is present inside the _____ directory.
3. The _____ statement is used to make a component available for use by other components within the application.

State if the statements are True or False.

4. To make a component appear on the output, it needs to be imported into the App.js file.

   _____

5. To assign a class in a React application, the 'className' attribute is used.

   _____

6. To use CSS flexbox, we need to set the 'position' property to 'flex'.

   _____

Answer the following questions in one line.

7. What are Components in React?

   _____

   _____

8. What is JSX?

   _____

   _____

Tech Flash

React : It is a widely popular component-based JavaScript library developed by Facebook, primarily used for building user interfaces.

Node.js : It is an open-source, cross-platform runtime environment that enables JavaScript code to be executed on the server.

Flexbox : It is a layout model in CSS that allows the flexible arrangement of items within a container.